

ANALYTICAL PROBLEM SOLVING

Warehouse Delivery Robot – Search Problem

a) PEAS Description

PEAS stands for **Performance measure, Environment, Actuators, and Sensors**.

Component	Description
Performance Measure	Deliver the package to (5,5) with minimum number of moves/cost, avoid blocked cells, and reach the destination safely.
Environment	A 5×5 warehouse grid containing blocked cells: (2,2), (2,3), (3,3), (4,2), (4,4).
Actuators	Move Up, Down, Left, or Right.
Sensors	Detect the robot's current position, nearby cells, obstacles, and destination.

b) Type of Intelligent Agent

The best-suited agent is a **Goal-Based Agent**.

Reason:

The robot has a specific goal: **reach cell (5,5) and deliver the package**. It must consider possible movements and select a sequence of actions that leads from the initial position (1,1) to the goal while avoiding blocked cells.

A simple reflex agent is not suitable because the robot needs to plan a sequence of movements rather than react only to the current situation.

c) Formal Search Problem

1. Initial State

$\backslash$ [Initial \ State = (1,1) \]

2. Goal State

$\backslash$ [Goal \ State = (5,5) \]

3. Actions

The robot can perform:

$\text{Actions} = \{\text{Up, Down, Left, Right}\}$

An action is allowed only if the resulting cell is within the 5×5 grid and is not blocked.

4. Transition Model

The transition model gives the new position after an action.

For example:

$(1,1) + \text{Down} = (2,1)$ $(1,1) + \text{Right} = (1,2)$

Blocked cells cannot be entered.

5. Path Cost

Each movement has a cost of 1.

$\text{Path Cost} = \text{Number of moves}$

Therefore, a path containing 8 moves has a total cost of:

$8 \times 1 = 8$

d) Breadth-First Search (BFS)

Action order: Down $\rightarrow$ Right $\rightarrow$ Up $\rightarrow$ Left

BFS uses a **FIFO queue**.

Step-by-step expansion

Step Expanded Node Frontier / Queue after expansion

1	(1,1)	[(2,1), (1,2)]
2	(2,1)	[(1,2), (3,1)]
3	(1,2)	[(3,1), (1,3)]
4	(3,1)	[(1,3), (4,1), (3,2)]
5	(1,3)	[(4,1), (3,2), (1,4)]
6	(4,1)	[(3,2), (1,4), (5,1)]
7	(3,2)	[(1,4), (5,1)]
8	(1,4)	[(5,1), (2,4), (1,5)]
9	(5,1)	[(2,4), (1,5), (5,2)]

Step Expanded Node Frontier / Queue after expansion

10	(2,4)	[(1,5), (5,2), (3,4), (2,5)]
11	(1,5)	[(5,2), (3,4), (2,5)]
12	(5,2)	[(3,4), (2,5), (5,3)]
13	(3,4)	[(2,5), (5,3), (3,5)]
14	(2,5)	[(5,3), (3,5)]
15	(5,3)	[(3,5), (5,4), (4,3)]
16	(3,5)	[(5,4), (4,3), (4,5)]
17	(5,4)	[(4,3), (4,5), (5,5)]
18	(4,3)	[(4,5), (5,5)]
19	(4,5)	[(5,5)]
20	(5,5) Goal	[]

BFS Final Path

$\backslash [(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (5,2) \rightarrow (5,3) \rightarrow (5,4) \rightarrow (5,5)]$

Total Cost

Number of moves = **8**

$\backslash \boxed{\text{Cost} = 8} \backslash$

Since every move has equal cost, BFS finds the **shortest path**.

e) Depth-First Search (DFS)

DFS uses a **LIFO stack**.

Using the same action preference, **Down is explored first**.

Step-by-step expansion**Step Expanded Node Stack after expansion**

1	(1,1)	[(1,2), (2,1)]
2	(2,1)	[(1,2), (3,2), (4,1)]

Step Expanded Node Stack after expansion

3	(3,1)	[(1,2), (3,2), (4,1)]
4	(4,1)	[(1,2), (3,2), (5,1)]
5	(5,1)	[(1,2), (3,2), (5,2)]
6	(5,2)	[(1,2), (3,2), (4,3), (5,3)]
7	(5,3)	[(1,2), (3,2), (4,3), (5,4)]
8	(5,4)	[(1,2), (3,2), (4,3), (5,5)]
9	(5,5) Goal	—

DFS Final Path

$\backslash [(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (5,2) \rightarrow (5,3) \rightarrow (5,4) \rightarrow (5,5)]$

Total Cost

$\backslash \boxed{\text{Cost} = 8} \backslash$

In this particular grid and with this node ordering, **DFS also finds the shortest path**. However, this is not guaranteed for DFS in general.

f) BFS vs DFS Comparison

Feature	BFS	DFS
Completeness	Complete	Complete for this finite grid with visited-state checking
Optimality	Optimal when all step costs are equal	Not guaranteed to be optimal
Time Complexity	$\backslash (O(b^d)) \backslash$	$\backslash (O(b^m)) \backslash$
Space Complexity	$\backslash (O(b^d)) \backslash$	$\backslash (O(bm)) \backslash$
Data Structure	Queue (FIFO)	Stack (LIFO)
Path in this problem	8 moves	8 moves

Feature	BFS	DFS
Best use	Shortest path problems	Deep exploration when memory is limited

Where:

- b = branching factor
- d = depth of the shallowest solution
- m = maximum search depth

PROGRAM

 i3.py - C:/Users/sridh/AppData/Local/Programs/Python/Python314/i3.py (3.14.7)

File Edit Format Run Options Window Help

```
from collections import deque

# 5x5 grid
rows, cols = 5, 5

start = (1, 1)
goal = (5, 5)
|
blocked = {(2, 2), (2, 3), (3, 3), (4, 2), (4, 4)}

# Possible movements
moves = [
    (1, 0),      # Down
    (0, 1),      # Right
    (-1, 0),     # Up
    (0, -1)      # Left
]

def valid(cell):
    r, c = cell
    return 1 <= r <= rows and 1 <= c <= cols and cell not in blocked

# ----- BFS -----
def bfs():
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        current, path = queue.popleft()

        if current == goal:
            return path

        for dr, dc in moves:
            next_cell = (current[0] + dr, current[1] + dc)

            if valid(next_cell) and next_cell not in visited:
                visited.add(next_cell)
                queue.append((next_cell, path + [next_cell]))
```

```

        queue.append((next_cell, path + [next_cell]))

    return None

# ----- DFS -----
def dfs():
    stack = [(start, [start])]
    visited = {start}

    while stack:
        current, path = stack.pop()

        if current == goal:
            return path

        # Reverse order because stack is LIFO
        for dr, dc in reversed(moves):
            next_cell = (current[0] + dr, current[1] + dc)

            if valid(next_cell) and next_cell not in visited:
                visited.add(next_cell)
                stack.append((next_cell, path + [next_cell]))

    return None

# Run BFS
bfs_path = bfs()
print("BFS Path:")
print(bfs_path)
print("BFS Cost:", len(bfs_path) - 1)

# Run DFS
dfs_path = dfs()
print("\nDFS Path:")
print(dfs_path)
print("DFS Cost:", len(dfs_path) - 1)

```

OUTPUT :

```

>>> ===== RESTART: C:/Users/sridh/AppData/Local/Programs/Python/Python314/i3.py =====
BFS Path:
[(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)]
BFS Cost: 8

DFS Path:
[(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)]
DFS Cost: 8
>>>

```

Conclusion

BFS is more suitable for this warehouse delivery problem because every movement has a uniform cost of 1 and the objective is to reach the destination using the **minimum number of moves**. BFS guarantees the shortest path, whereas DFS may find a longer path depending on the order in which nodes are explored.

Final Answer

$\boxed{\text{BFS is the preferred algorithm for this warehouse robot.}}$